# Repaso Parcial I — Cine /2026

***Programación II — UTN FRC (TUP)***

*Material de repaso para el recuperatorio. Mismo tipo de ejercicio que el Parcial I, con otro dominio y otros números.*

## Índice


1. Enunciado
2. Se pide
3. Servicios
4. Reglas de negocio
5. Importante
6. Tabla de puntajes


## 1. Enunciado

Una cadena de cines quiere un sistema simple para administrar sus salas y las funciones que se proyectan en cada una.

Una **Sala** puede tener muchas **Funciones** (cero o más). Cada **Función** pertenece obligatoriamente a una única **Sala**.

Cada sala tiene un nombre, un formato de proyección ( `2D` , `3D` o `IMAX` ) y un estado que indica si está activa o fuera de servicio.

Cada función tiene la película que se proyecta, su duración en minutos, la fecha en la que se programó, un estado que indica si ya finalizó, y la sala a la que pertenece.

El proyecto base (modelos y `DbContext` ) ya está resuelto — no hace falta Data First ni Code First. Se pide completar el **repositorio** y el **controller** de la API.

## 2. Se pide

Completar `SalaRepository` (que implementa `IRepository` ) y `SalasController` para que el sistema permita:

- Consultar una sala por su identificador.
- Consultar las salas activas que tienen funciones pendientes, pudiendo filtrar por formato.
- Asignar (programar) una nueva función a una sala, respetando las reglas de negocio.

## 3. Servicios

| Servicio | Código | Cuándo |
|---|---|---|
| `GET /api/salas/{id}` | `200 OK` | la sala existe, devuelve sus datos |
| `GET /api/salas/{id}` | `404 Not Found` | la sala no existe |
| `GET /api/salas/activas` | `200 OK` | siempre — con la lista (puede venir vacía) de salas activas con funciones pendientes; acepta un parámetro opcional por URL `formato` para filtrar (sin distinguir mayúsculas/minúsculas) |
| `POST /api/salas/{id}/funcion` | `201 Created` | se pudo programar la función |
| `POST /api/salas/{id}/funcion` | `400 Bad Request` | alguna regla de negocio lo impide |

El cuerpo de `POST /api/salas/{id}/funcion` es un JSON con la película y los minutos de duración:

```
{
  "pelicula": "Nombre de la película",
  "minutos": 120
}
```

## 4. Reglas de negocio (asignación de una función)

1. La sala debe existir **y** estar activa ( `EstaActiva == true` ).
2. La duración tiene que ser mayor a 0 y menor o igual a 240 minutos.
3. Si la sala ya tiene 4 o más funciones **pendientes** ( `EstaFinalizada == false` ), no se pueden programar funciones nuevas (límite alcanzado).
4. Si se cumplen las tres reglas anteriores: se crea la función con `EstaFinalizada = false` y `Fecha` = la fecha actual (UTC), se guarda en la base, y el método devuelve `true` . Si alguna regla falla, el método devuelve `false` (sin guardar nada) y el `Controller` traduce eso en un `400 Bad Request` .

## 5. Importante

- Respetar los nombres de los métodos y variables ya declarados en `IRepository` y en `SalasController` — la corrección se apoya en esos nombres exactos.
- No hace falta tocar `Program.cs` , `appsettings.json` ni los modelos — ya están resueltos.
- Antes de entregar, probar los casos del archivo `RepasoCineWebAPI.http` (dentro del proyecto) y confirmar que cada uno da el código que indica el comentario.
- Si algo no queda claro, revisar primero `GUIA-ERRORES-FRECUENTES.md` — tiene pistas sobre los errores más comunes de este tipo de ejercicio, sin dar la solución.

## 6. Tabla de puntajes

| Ítem | Puntos |
|---|---|
| Abstracción de datos (uso correcto del modelo / `DbContext` provisto) | 25 |
| Consultar Sala por Id | 20 |
| Consultar salas con funciones pendientes | 25 |
| Asignar función a una Sala | 20 |
| Validaciones de negocio en la asignación | 10 |
| **Total** | **100** |

---

*Éxitos Programación II.*